

When does a vision-language model stop needing the image?

Phase 3: why Balanced Token Pruning breaks text reading on Qwen2.5-VL

P. S. Kedar | Supervised by Prof. Divya Saxena | Mentored by Aditya Sharma, PhD Scholar

IIT Jodhpur

Three terms we use throughout

Baseline



The original model, no pruning.
Every visual token is kept.

It is the reference: every other result is compared against it.

Pruning



Remove some visual tokens, keep
the rest.

Fewer tokens to process, so the model
runs faster.

Deletion



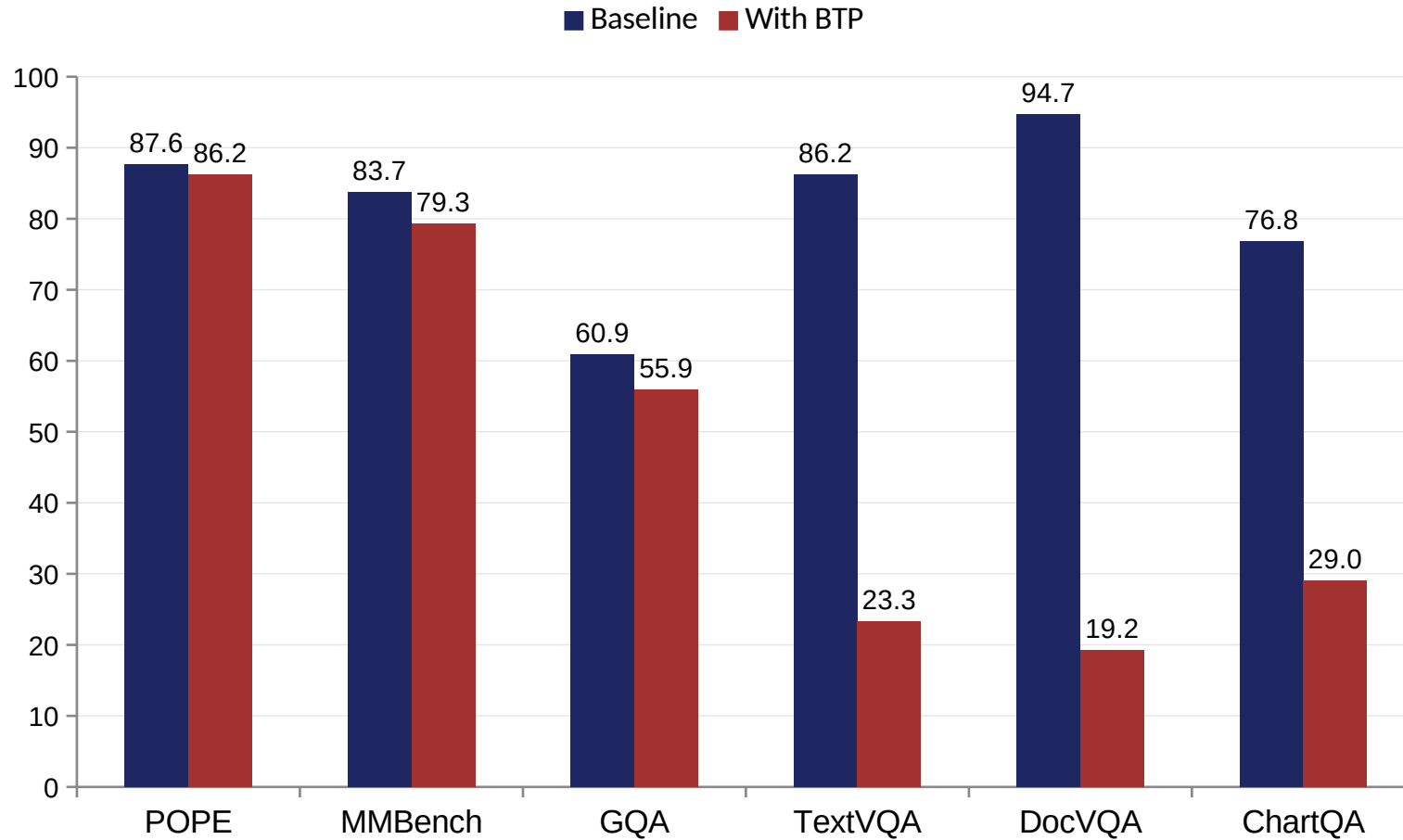
Remove all remaining visual tokens
at one layer.

From that layer onward, the model
cannot see the image at all.

Pruning and deletion are not the same thing. That difference is the whole story of this phase.

1. WHAT WE OBSERVED

Qwen stays fine on general tasks, but stops reading



General tasks

Lose only a few points.

Reading tasks

Collapse. TextVQA falls from 86.2% to 23.3%.

The paper reports only general tasks, so it would not show this.

Four possible explanations

1

Wrong tokens chosen

BTP keeps the wrong visual tokens and throws away the text.

2

Text looks redundant

Text patches look alike, so BTP treats them as duplicates.

3

Too much removed

Simply too many tokens are thrown away for reading to work.

4

Text not protected

If we made sure every text token survived, reading would come back.

3. WHAT THOSE EXPERIMENTS TOLD US

All four were ruled out

Explanation	What we tested	What we got	What it tells us
Wrong tokens chosen	Replaced BTP's token choice with other rules, including random	All scored 17 to 19% on TextVQA	Which tokens are kept does not matter here
Text looks redundant	Compared how similar text patches are to background patches	Text patches were less alike, not more (0.38 vs 0.44)	Our starting idea was wrong
Too much removed	Gradually reduced the tokens kept, from 100% down to 12.5%	At 90% kept, TextVQA had already dropped to the same low level as at 12.5%	The collapse is not just about removing too many
Text not protected	Forced every text token to survive, using the true text locations	No improvement: 19.7% with, 19.7% without	Even a perfect choice of tokens does not fix it

Four unrelated changes, and the score barely moved. That pointed somewhere else.

So where is the loss happening?

WHAT WE TESTED

We kept every visual token through all of BTP's pruning stages, so nothing was pruned at all. If pruning were the cause, performance should come back to the baseline.

WHAT WE GOT

Condition	TextVQA
Baseline	87.6%
BTP, but keeping 100% of tokens	17.8%

Nothing pruned, and it still collapsed.

WHAT IT TELLS US

The damage is not coming from the pruning stages.

Something after them is removing the image. That is where we looked next.

The code removes every remaining visual token at one layer



Pruning keeps 12.5% of the visual tokens. Then, at a later layer, the released code deletes all of them.

For Qwen, the paper describes keeping 12.5% at this last stage. The released implementation removes everything instead. This is a difference between the description and the code we tested; it says nothing about why it is there.

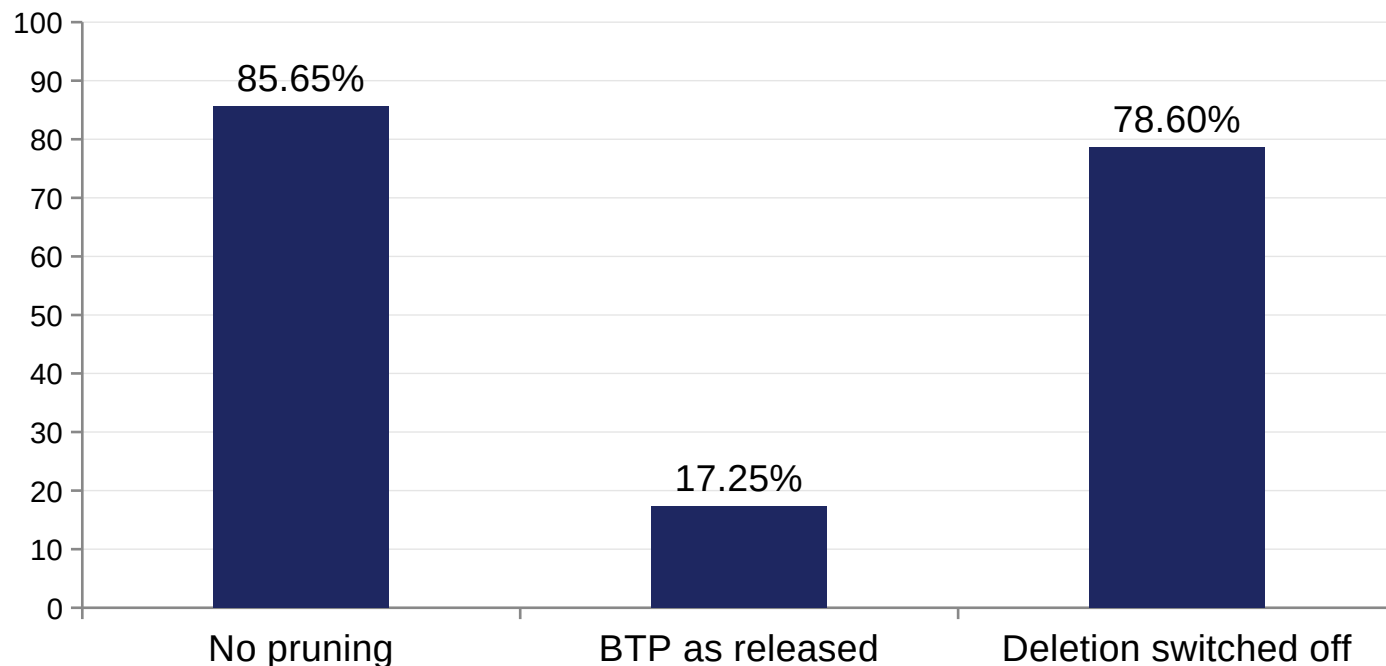
6. THE INTERVENTION

Does that deletion actually cause the collapse?

WHAT WE TESTED

We kept BTP's 12.5% pruning exactly as it is, and switched off only the final step that deletes the remaining tokens.

WHAT WE GOT



WHAT IT TELLS US

Switching off that one step brings TextVQA from 17% back to 79%.

The collapse on Qwen is mainly caused by removing all remaining tokens, not by the earlier pruning.

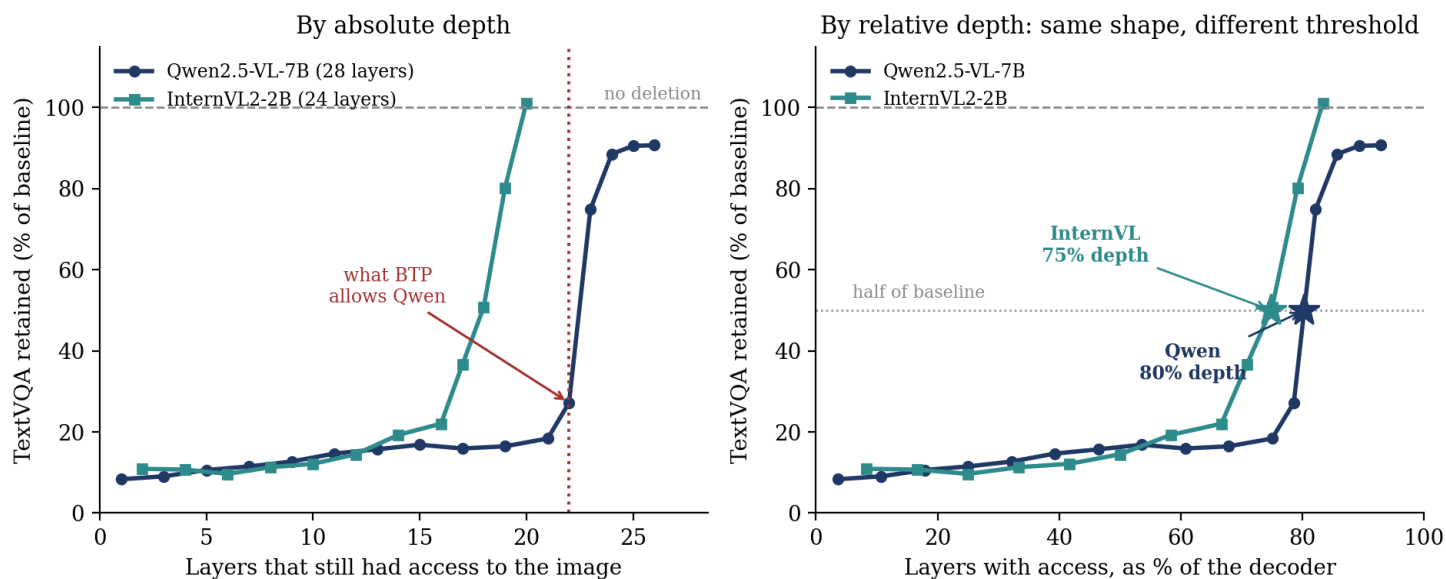
7. MOVING THE DELETION POINT

Is that layer special, or just too early?

WHAT WE TESTED

We moved the point of complete deletion through the model, from early layers to late ones, and measured TextVQA each time. On two models.

WHAT WE GOT



WHAT IT TELLS US

Delete too early and reading collapses.
Delete late enough and it barely hurts.

The change is sudden, not gradual. There is a threshold.

Both models show the same pattern, at different depths.

Threshold and depth, in plain terms

Threshold

How many layers the model needs to see the image for, to keep at least half of its original TextVQA score.

Depth %

The same threshold as a share of the model. How far through the model the image needs to stay visible.

Qwen2.5-VL

22.5 of 28 layers
 $22.5 / 28 \times 100$

80.3%

InternVL2

18.0 of 24 layers
(17.95 before rounding)

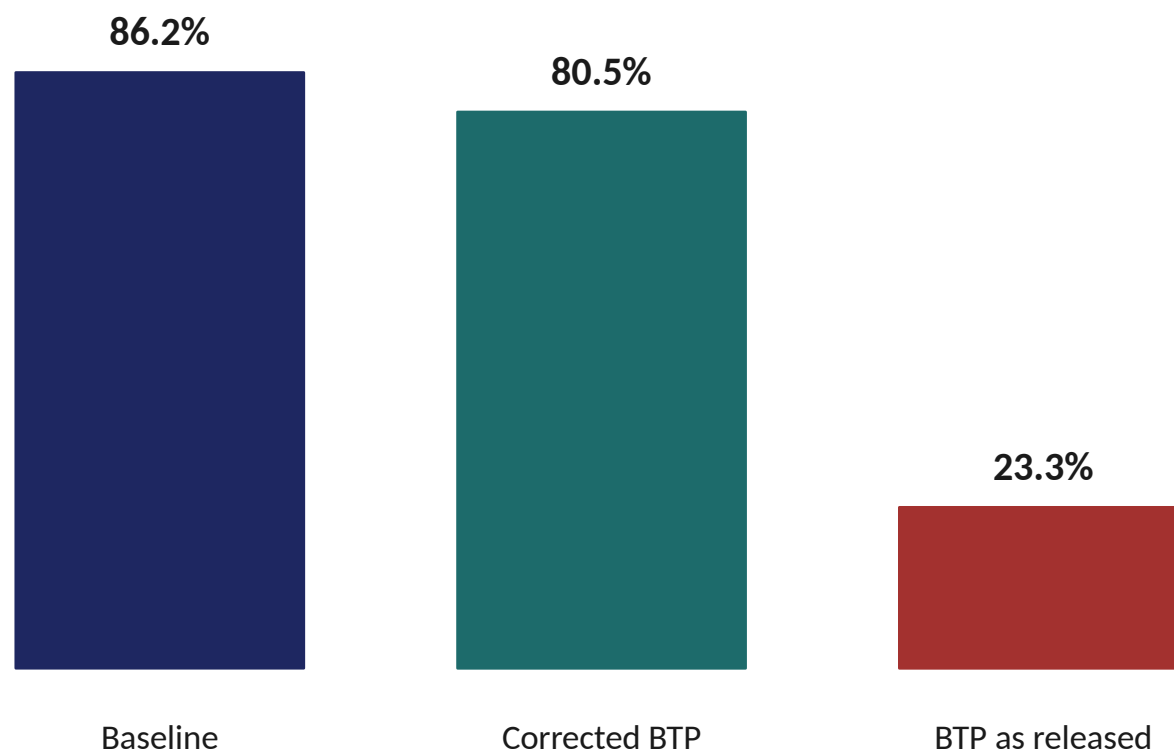
74.8%

Measured on TextVQA, so these are TextVQA-specific. BTP's Qwen setting keeps the image for 22 layers, just short of 22.5.

9. WHAT ACTUALLY CAUSES THE LOSS?

Two separate costs, worked through on TextVQA

Corrected BTP = BTP with only the final deletion switched off. It still prunes to 12.5%.



Pruning cost

baseline – corrected BTP
 $86.2 - 80.5 = 5.7$ points

Deletion cost

corrected BTP – released BTP
 $80.5 - 23.3 = 57.2$ points

On TextVQA, almost all of the loss comes from deletion.

10. DOES THIS HOLD ACROSS TASKS?

Which cost dominates depends on the task

Task	Pruning cost	Deletion cost	Dominant
TextVQA	5.7	57.2	deletion
DocVQA	17.9	57.6	deletion
ChartQA	31.4	16.4	pruning
AI2D	7.2	0.4	pruning
GQA	2.1	2.9	both small
POPE	1.3	0.1	both small
MMBench	4.7	0.3	both small

WHAT IT TELLS US

Dominant = whichever caused the bigger share of the drop.

On TextVQA and DocVQA, deletion does most of the damage.

On ChartQA, pruning itself costs twice as much as deletion.

So the finding is not simply "deletion is bad". The main source of loss depends on the task.

Its benchmarks cannot see the reading loss

Share of baseline kept	Paper's benchmarks	Reading tasks
Qwen, BTP as released	96.1%	28.4%
Qwen, deletion switched off	96.9%	77.9%
InternVL, pruning only	94.9%	56.5%

On the paper's benchmarks, every row looks healthy.

On reading, the same configurations lose between a quarter and three quarters of their ability.

InternVL has no deletion step at all, and the gap is still there.

A benchmark set with no reading task cannot tell a model that reads from one that does not.

Three separate findings

1

Implementation

In the Qwen configuration we tested, removing all remaining visual tokens causes most of the reading collapse.

2

Model

Each model needs to see the image up to a measurable depth: about 80% of Qwen, 75% of InternVL, on TextVQA.

3

Evaluation

Standard benchmarks can hide large losses on reading tasks, with or without the deletion step.

These stand separately. The third holds even if the first did not.

Limits, and what comes next

What we would not claim

- Two models only. The shared pattern is worth testing further, not a rule
- Thresholds come from TextVQA. Other tasks may differ
- The InternVL pruning is our own, simplified version
- Results use 500-question subsets per task

Next

- Measure the threshold on ChartQA and DocVQA, ChartQA first
- Repeat on a third model
- Prepare the manuscript